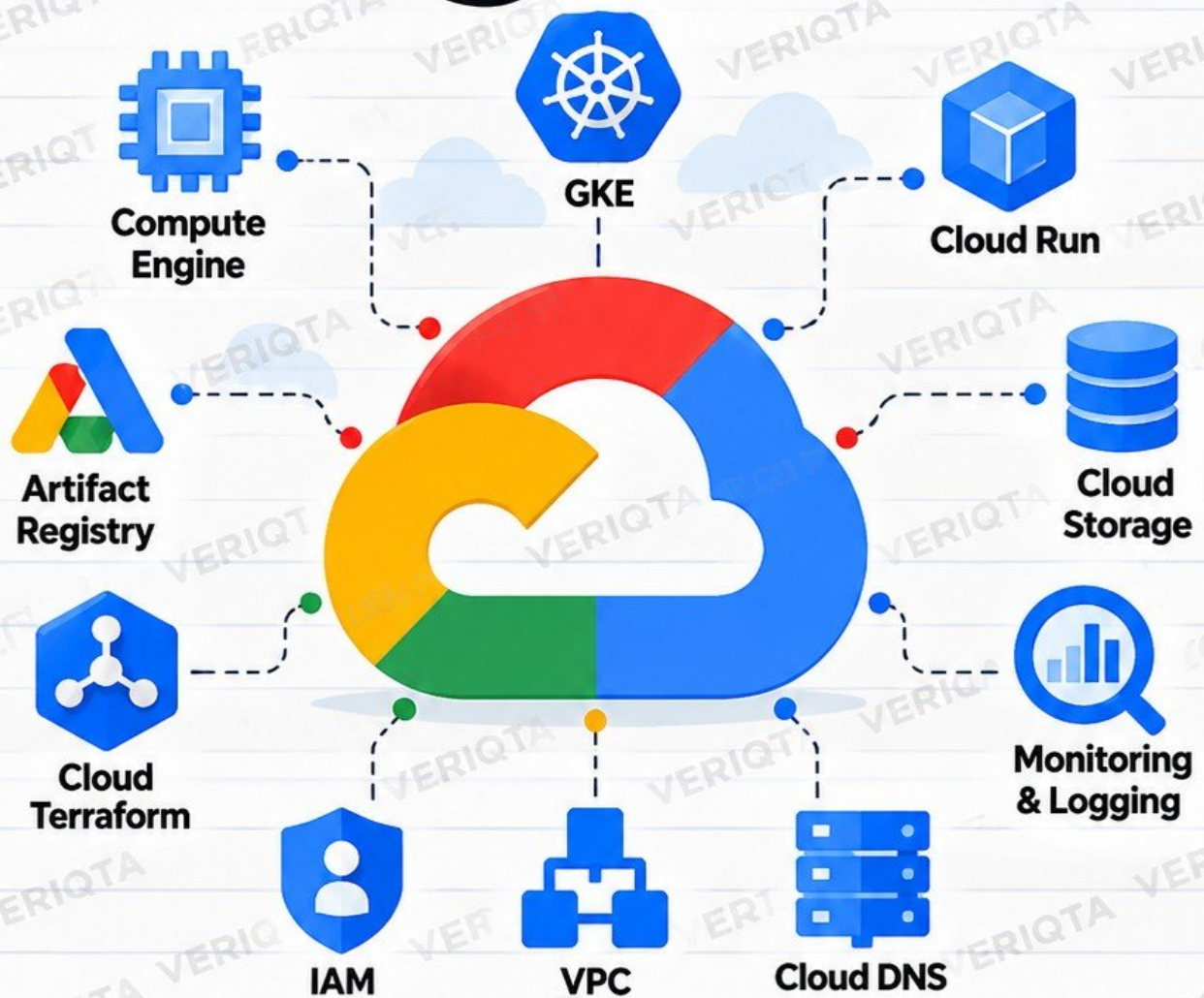


GCP

for DevOps Engineers



01 GCP Through a DevOps Engineer's Eyes

What is Google Cloud Platform (GCP)?

- **GCP** is a cloud computing platform provided by **Google**.
- It offers infrastructure, platform and serverless services to build, deploy and operate applications.
- **DevOps** engineers use **GCP** to run applications, automate infrastructure, manage containers, build **CI/CD** pipelines, and monitor systems at scale.

Why DevOps Engineers Use GCP

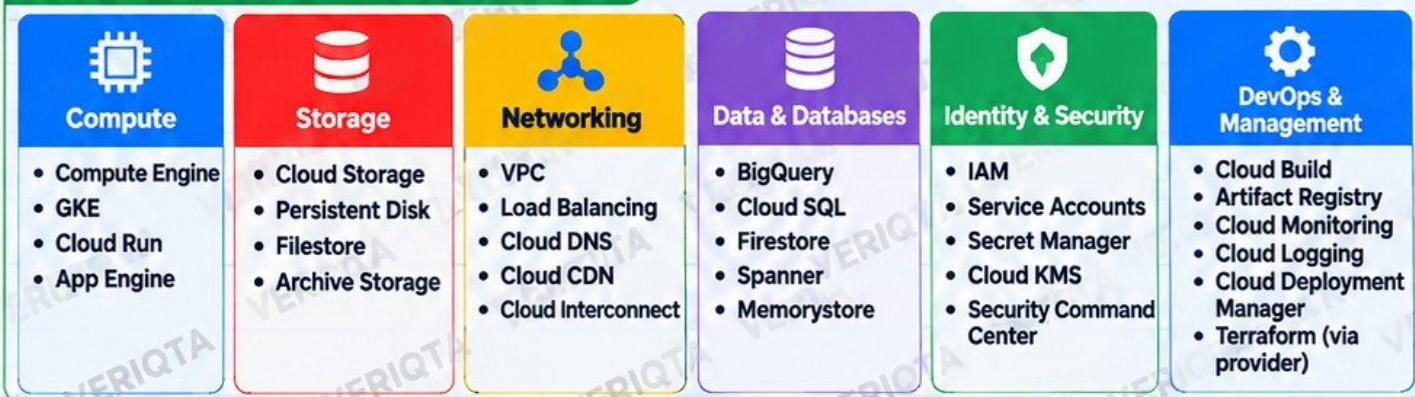
- Reliable and high-performance cloud infrastructure.
- Strong support for containers and Kubernetes (**GKE**).
- Serverless options like **Cloud Run**.
- Integrated **CI/CD** with **Cloud Build**.
- Good observability, security and IAM.
- Global network with low latency.
- Pay only for what you use.

Global Infrastructure: Regions and Zones

- A **Region** is a specific geographic location (for example us-central1).
- A Region contains multiple **Zones**.
- A **Zone** is an isolated location within a region (for example us-central1-a).
- You choose regions and zones to reduce latency and increase availability for your applications.



Core Service Categories in GCP



Where GCP Fits in a DevOps Workflow



Key Takeaway

- **GCP** gives DevOps engineers the tools to build, deploy, scale and operate applications reliably.
- You work with regions, zones and many integrated services to automate the entire lifecycle.

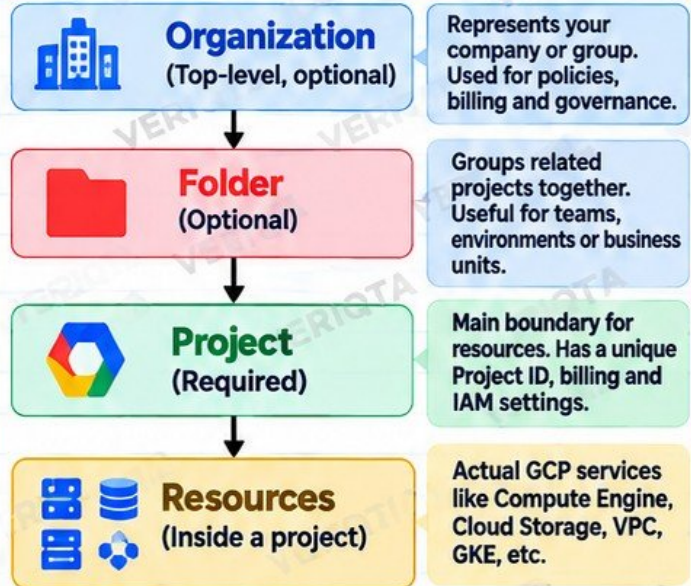
What is the Resource Hierarchy?

- GCP organizes resources in a hierarchy: **Organization** → **Folder** → **Project** → **Resources**.
- This hierarchy helps you manage access, billing, policies and security at scale.
- Projects are the main boundary for most resources and services.

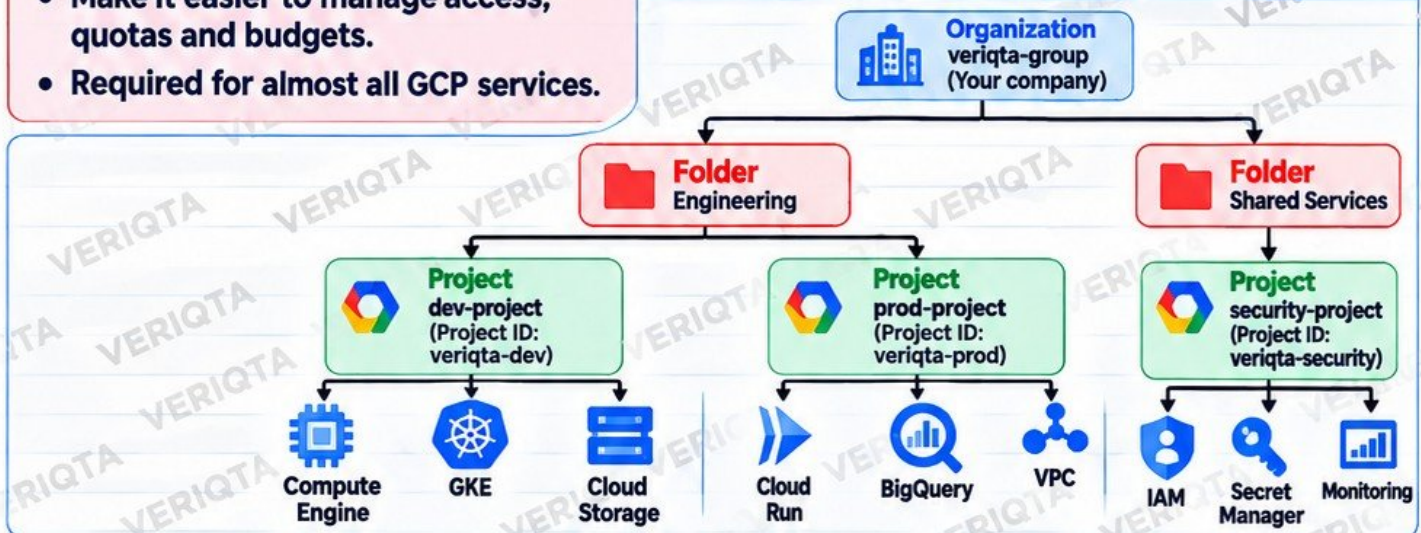
Why Projects are Important

- Provide a boundary for resources.
- Have their own IAM policies.
- Are linked to a billing account.
- Help separate environments (dev, test, prod).
- Make it easier to manage access, quotas and budgets.
- Required for almost all GCP services.

Hierarchy Structure in GCP



Example Hierarchy (Real World)



Project ID, Project Name and Project Number

Item	Description
Project ID	Unique ID used in commands and APIs. Example: veriqta-dev
Project Name	Friendly name you choose. Example: Dev Environment
Project Number	Automatically generated by Google. Example: 123456789012

Billing Relationship

- A project must be linked to a **billing account** to use most GCP services.
- You can link multiple projects to the **same billing account**.
- Billing can be managed at the **organization level** or the **project level**.
- Projects help you **track and control costs** by environment, team or application.

Common Use Cases

- Separate projects for dev, test and prod.
- Different projects for different teams.
- Dedicated project for security and shared services.
- Use folders to organize many projects.
- Apply IAM and policies at the organization or folder level.



Key Takeaways

- Projects are the main boundary in GCP.
- They control IAM, billing and access.
- Use folders and organizations for governance.
- Start with a clear project structure.
- Keep environments separate for security and cost control.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

03 IAM and Service Accounts

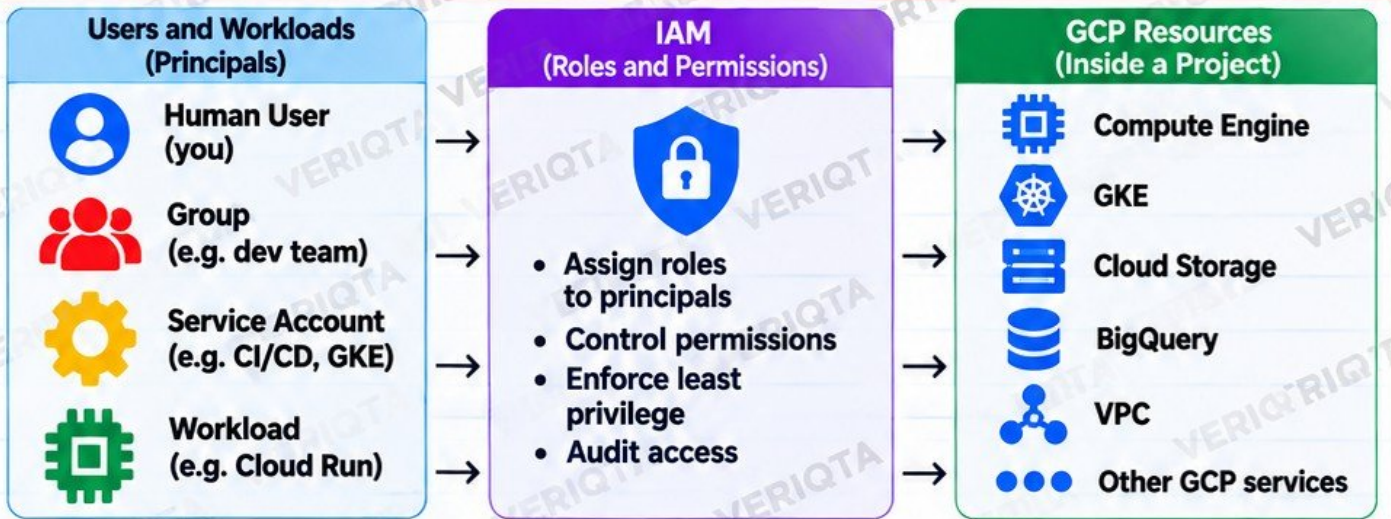
What is IAM?

- IAM (Identity and Access Management) controls **who** can access Google Cloud resources and **what** they can do.
- It uses principals, roles and permissions.
- It helps you apply least privilege and keep your cloud environment secure.

Key Concepts

	Principals	A user, group, service account or workload that can be given access to resources.
	Roles	A collection of permissions. You assign roles to principals instead of individual permissions.
	Permissions	Specific actions allowed or denied (for example, compute.instances.get).
	Service Accounts	Special Google accounts for applications and workloads instead of human users.

How IAM Fits into a DevOps Environment



Service Accounts in DevOps

- Used by applications, CI/CD pipelines, Kubernetes clusters and other workloads.
- Can be assigned specific roles (**least privilege**).
- Authenticate using a **key file** or, preferably, **Workload Identity** (no key file).
- Helps automate tasks without using a human account.

Example: Create a Service Account

```
gcloud iam service-accounts create cicd-sa \
  --display-name="CI/CD Service Account"

gcloud projects add-iam-policy-binding my-project \
  --member="serviceAccount:cicd-sa@my-project.iam.gserviceaccount.com" \
  --role="roles/storage.admin"
```

Best Practices

- ✓ Use least privilege (only the permissions needed).
- ✓ Use service accounts for automation.
- ✓ Prefer Workload Identity over key files.
- ✓ Avoid using owner roles for daily work.
- ✓ Review IAM access regularly.
- ✓ Use separate service accounts for different workloads.

Common Mistakes

- ✗ Giving too many permissions (over-permissioning).
- ✗ Using the owner role for convenience.
- ✗ Storing service account key files in Git.
- ✗ Using human accounts for automated tasks.
- ✗ Not rotating or reviewing service account access.



Key Takeaway

- IAM protects your GCP environment by controlling who can access what.
- Use service accounts for workloads and follow **least privilege**.
- Never treat credentials casually.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

04 The GCP Network Mental Model

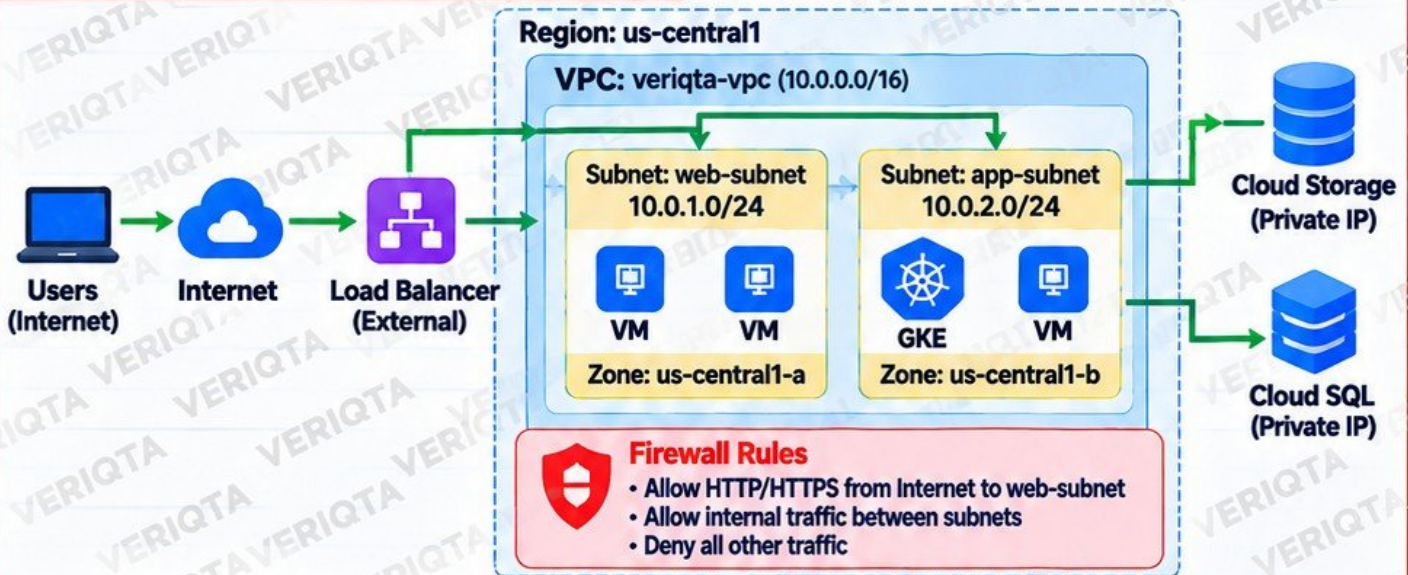
What is GCP Networking?

- GCP provides a **global**, high-performance network to connect your applications, users and services.
- You use **VPC**, **subnets**, **routes** and **firewall rules** to control traffic.
- It is designed to be **secure**, **scalable** and **highly available**.

Key Components

	Regions	Geographic locations around the world (e.g. us-central1).
	Zones	Isolated locations within a region (e.g. us-central1-a).
	VPC	Your private, isolated network in GCP.
	Subnets	Ranges of IP addresses within a VPC.
	Routes	Rules that tell traffic where to go.
	Firewall Rules	Allow or deny traffic based on rules.
	IP Addresses	Internal (private) and external (public) IPs.

GCP Network Architecture (Example)



VPC and Subnet CIDR Example

Component	Example	Description
VPC	10.0.0.0/16	Total IP range for your VPC.
Web Subnet	10.0.1.0/24	IP range for web servers.
App Subnet	10.0.2.0/24	IP range for application servers.
Database Subnet	10.0.3.0/24	IP range for databases.

How Traffic Flows

- User sends a request to your application (e.g. www.example.com).
- Traffic arrives at an external Load Balancer.
- Load Balancer forwards traffic to VMs or GKE Pods in your VPC (web-subnet).
- Firewall rules allow the traffic based on your rules.
- Application communicates with other services (e.g. Cloud SQL, Cloud Storage) using internal IPs.

Common Firewall Rules

Rule	Direction	Action	Example
Allow HTTP/HTTPS	Ingress	Allow	0.0.0.0/0 → tcp:80,443
Allow SSH (restricted)	Ingress	Allow	My IP → tcp:22
Allow Internal Traffic	Ingress	Allow	10.0.0.0/16 → all
Deny All Others	Ingress	Deny	All IPs → all

Key Takeaways

- A VPC is your private network in GCP.
- Subnets divide IP ranges within a VPC.
- Regions and zones provide high availability.
- Firewall rules control which traffic is allowed.
- Use internal IPs for communication between services.
- Design your network to be secure and least-privilege.

05 Compute Engine, Your First VM

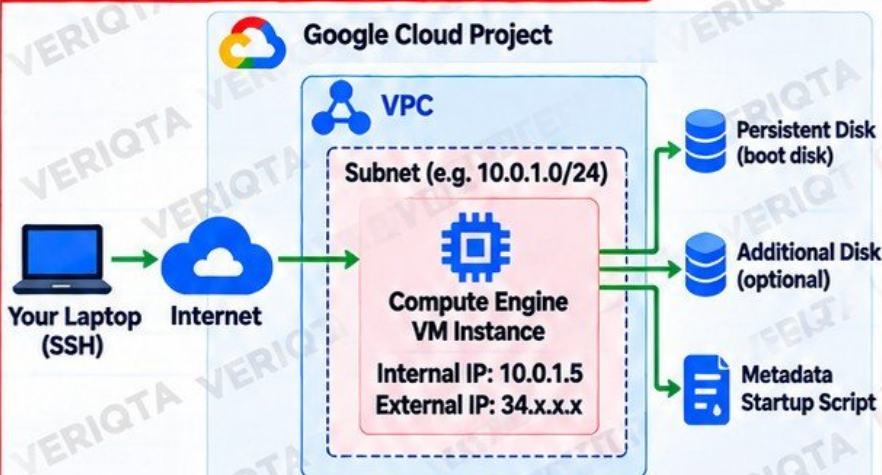
What is Compute Engine?

- Compute Engine provides **virtual machines (VMs)** in Google Cloud.
- You can choose machine types, images, disks, networking and security settings.
- It gives you full control over the OS and software, similar to a physical server, but runs in Google's cloud.
- Commonly used for application servers, bastion hosts, development environments and self-managed workloads.

Key Components of a VM Instance

	Machine Type	Defines CPU, memory and performance (e.g. e2-medium).
	Image	Operating system for the VM (e.g. Ubuntu, Debian, Windows).
	Disk	Boot disk and optional additional disks (Persistent Disk).
	Network	VPC, subnets, internal and external IP addresses.
	Firewall	Rules that allow or deny traffic (e.g. SSH, HTTP).
	Metadata & Startup Scripts	Pass configuration data or run scripts when the VM starts.

How a Compute Engine VM Works



Create a VM (gcloud CLI)

```
gcloud compute instances create dev-vm \
  --zone=us-central1-a \
  --machine-type=e2-medium \
  --image-family=ubuntu-2204-lts \
  --image-project=ubuntu-os-cloud \
  --boot-disk-size=20GB \
  --tags=http-server,ssh
```

What this does:

- Creates a VM named dev-vm
- In zone us-central1-a
- Using e2-medium machine type
- With Ubuntu 22.04 image
- 20GB boot disk
- Adds firewall tags

Connect to the VM (SSH)

```
gcloud compute ssh dev-vm \
  --zone=us-central1-a
```

What this does:

- Opens an SSH session to the VM using your Google Cloud credentials.
- No need to manage SSH keys manually.

View VM Details

```
gcloud compute instances describe \
  dev-vm --zone=us-central1-a
```

What this shows:

- Machine type, image, disks
- Internal and external IP addresses
- Network and firewall tags
- Metadata and other settings

Stop, Start and Delete

```
gcloud compute instances stop dev-vm \
  --zone=us-central1-a

gcloud compute instances start dev-vm \
  --zone=us-central1-a

gcloud compute instances delete dev-vm \
  --zone=us-central1-a
```

Common Machine Types (Examples)

Type	Use Case	Key Features
e2-micro	Small dev/test VMs	Low cost, 1 vCPU, 1GB RAM
e2-medium	General purpose	2 vCPU, 4GB RAM
n2-standard	Compute intensive	High performance CPUs
n2-highmem	Memory intensive	More memory per CPU
c3	High performance	Latest generation, demanding apps

Lifecycle of a VM



Key Takeaways

- ✓ Compute Engine gives you full control over your VMs.
- ✓ Choose the right machine type and image for your workload.
- ✓ Use metadata and startup scripts to automate setup.
- ✓ Secure your VM with firewall rules.
- ✓ Start/stop VMs to manage cost.
- ✓ Delete unused VMs.
- ✓ Use it for dev, test or production workloads.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

06 Load Balancing and Application Traffic

What is Load Balancing?

- Load balancing **distributes client traffic** across multiple backends (VMs, GKE Pods or serverless services).
- It improves **availability, reliability and performance**.
- It performs **health checks** and only sends traffic to healthy backends.
- It can route traffic based on rules (host, path, region, etc).
- It is used for **global and regional** applications in GCP.

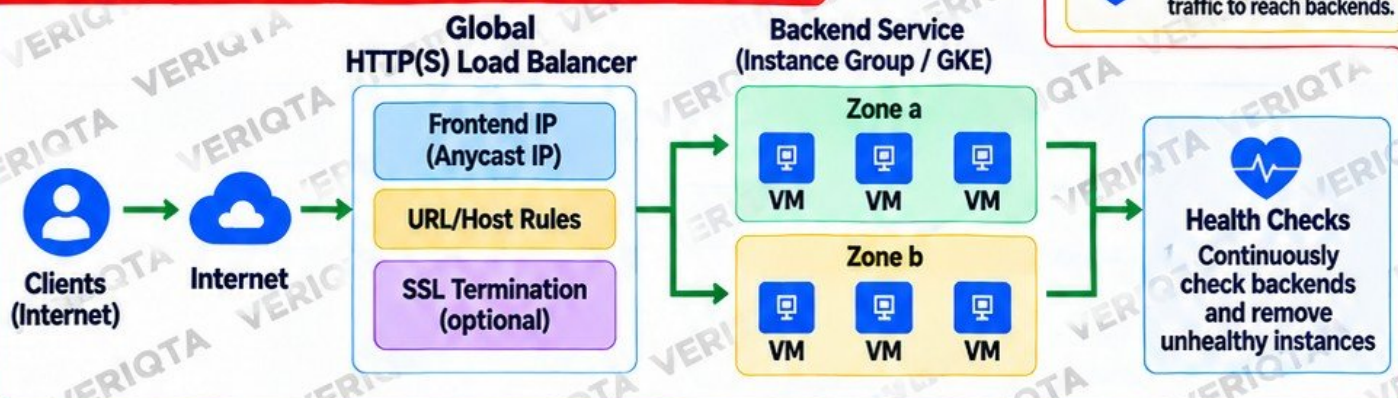
Types of Load Balancers in GCP

Type	Use Case	Key Features
Global HTTP(S) Load Balancer	Web applications (HTTP/HTTPS)	Global, anycast IP, CDN integration, URL routing, SSL
Global Internal Load Balancer	Internal web apps within VPC	Private IP, global internal access
Regional TCP/UDP Load Balancer	Non-HTTP apps (TCP/UDP)	Regional, L4 load balancing
Regional Internal TCP/UDP LB	Internal services within a region	Private IP, L4, low latency

Key Components

- Frontend**
Public or internal IP, protocol (HTTP/HTTPS/TCP).
- Backend Service**
Group of VMs, instance groups or GKE services.
- Health Check**
Checks if backends are healthy.
- URL or Host-based Routing**
Routes traffic based on rules (host, path, etc).
- Firewall Rules**
Allow load balancer traffic to reach backends.

How Load Balancing Works (HTTP/S Example)



Example: Create an HTTP(S) Load Balancer (gcloud CLI)

```
# Create a global static IP
gcloud compute addresses create web-lb-ip --global

# Create a health check
gcloud compute health-checks create http web-hc \
  --port 80 --request-path /

# Create a backend service and add an instance group
gcloud compute backend-services create web-backend \
  --protocol HTTP --health-checks web-hc --global
gcloud compute backend-services add-backend web-backend \
  --instance-group web-igm --instance-group-zone us-central1-a --global

# Create URL map, proxy and forwarding rule (simplified)
gcloud compute url-maps create web-map --default-service web-backend
gcloud compute target-http-proxies create http-proxy --url-map web-map
gcloud compute forwarding-rules create http-rule --global \
  --address web-lb-ip --target-http-proxy http-proxy --ports 80
```

Health Check Example

```
gcloud compute health-checks create http web-hc \
  --port 80 --request-path / \
  --check-interval 5s --timeout 5s --healthy-threshold 2 \
  --unhealthy-threshold 3
```

Typical Traffic Flow (HTTP/S)

- Client sends a request to the load balancer's IP or domain.
- Load balancer evaluates URL/host rules.
- Traffic is forwarded to a healthy backend.
- Health checks run continuously.
- If a backend is unhealthy, it is removed from rotation.
- Responses are returned to the client through the load balancer.

Common Routing Options

Type	Description
Host-based	Route based on domain (e.g. app.example.com).
Path-based	Route based on URL path (e.g. /api, /app).
Default backend	Used when no other rule matches.

Use Cases

- Web applications (HTTP/HTTPS).
- Microservices on GKE.
- Multiple environments (dev, test, prod).
- High availability and disaster recovery.
- Global applications with low latency.

Key Takeaways

- Load balancing distributes traffic to healthy backends.
- Use health checks to detect failures.
- Choose the right type of load balancer.
- Use URL/host rules for flexible routing.
- It improves availability, scalability and performance.

07 Cloud DNS and Name Resolution

What is Cloud DNS?

- Cloud DNS is Google Cloud's managed **Domain Name System (DNS)**.
- It translates **domain names** (e.g. **app.example.com**) to **IP addresses**.
- It supports **public and private** DNS zones.
- It is highly available, scalable and integrated with other GCP services.

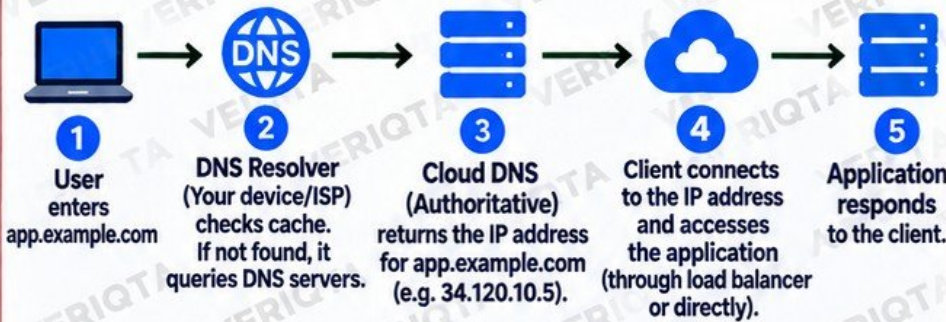
Key Concepts

Zone	Container for DNS records. Can be public or private.
Record	Maps a domain name to a value (IP, CNAME, etc).
Public Zone	Accessible from the internet.
Private Zone	Used inside a VPC for internal name resolution.
Name Resolution	Process of finding the IP address for a domain name.

Common Record Types

Type	Purpose
A	Maps a domain to an IPv4 address.
AAAA	Maps a domain to an IPv6 address.
CNAME	Alias to another domain name.
MX	Mail server record.
TXT	Arbitrary text (e.g. SPF).
NS	Name server record.
PTR	Reverse DNS (IP to name).

How DNS Resolution Works (Simplified Flow)



Public vs Private DNS

Public DNS Zone	- Accessible from the internet. - Used for public websites and services. - Example: example.com
Private DNS Zone	- Accessible only within your VPC or connected networks. - Used for internal services (e.g. database.internal). - Helps you avoid using public IPs for internal communication.

Example: Create a Cloud DNS Zone and Record (gcloud CLI)

```

# Create a public DNS zone
gcloud dns managed-zones create example-zone \
  --dns-name=example.com \
  --description="Public DNS zone for example.com"

# Create an A record for app.example.com
gcloud dns record-sets create app.example.com \
  --zone=example-zone \
  --type=A \
  --ttl=300 \
  --rrdatas=34.120.10.5
  
```

Example: Private DNS Zone (Internal)

```

# Create a private DNS zone for internal services
gcloud dns managed-zones create internal-zone \
  --dns-name=internal.example.com \
  --visibility=private \
  --networks=projects/PROJECT_ID/global/networks/your-vpc

# Add an internal record
gcloud dns record-sets create db.internal.example.com \
  --zone=internal-zone \
  --type=A \
  --ttl=300 \
  --rrdatas=10.0.1.10
  
```

Troubleshooting: Works by IP but Not by Hostname

Check	Command / Action	What to Look For
DNS resolution	<code>nslookup app.example.com</code> <code>dig app.example.com</code>	Does it return the correct IP?
Record exists	<code>gcloud dns record-sets list</code> <code>--zone=example-zone</code>	Is the record correct?
Correct zone	<code>gcloud dns managed-zones list</code>	Are you using the right zone (public or private)?
Propagation	Wait a few minutes or check TTL	Has the record propagated?
Client DNS	Check DNS server settings	Is the client using the expected DNS server (e.g. VPC DNS)?
Firewall / Network	Test connectivity to the IP	Can the client reach the IP?

Common Use Cases

- ✓ Host public websites and applications.
- ✓ Map custom domain names to load balancers.
- ✓ Internal name resolution for microservices.
- ✓ Separate DNS for dev, test and prod environments.
- ✓ Use private DNS for internal resources (databases, internal APIs).

Key Takeaways

- Cloud DNS translates domain names to IP addresses.
- ✓ Use public zones for internet-facing services.
- ✓ Use private zones for internal services.
- ✓ Choose the correct record type.
- ✓ DNS is often the reason an application works by IP but not by hostname.
- ✓ Always check DNS resolution, records, and network connectivity when troubleshooting.

08 Cloud Storage for DevOps

What is Cloud Storage?

- Cloud Storage is Google Cloud's **object storage** service.
- It stores and retrieves any amount of data such as files, artifacts, logs, backups and more.
- Data is stored in **buckets** as **objects**.
- It is durable, highly available, scalable and integrated with other GCP services.

Key Concepts

	Bucket	A container for objects.
	Object	A file stored in a bucket.
	Storage Class	Defines cost and availability.
	IAM Access	Controls who can access buckets/objects.
	Lifecycle Policy	Automatically manage objects (e.g. delete or move to cheaper class).
	Versioning	Keep multiple versions of an object.

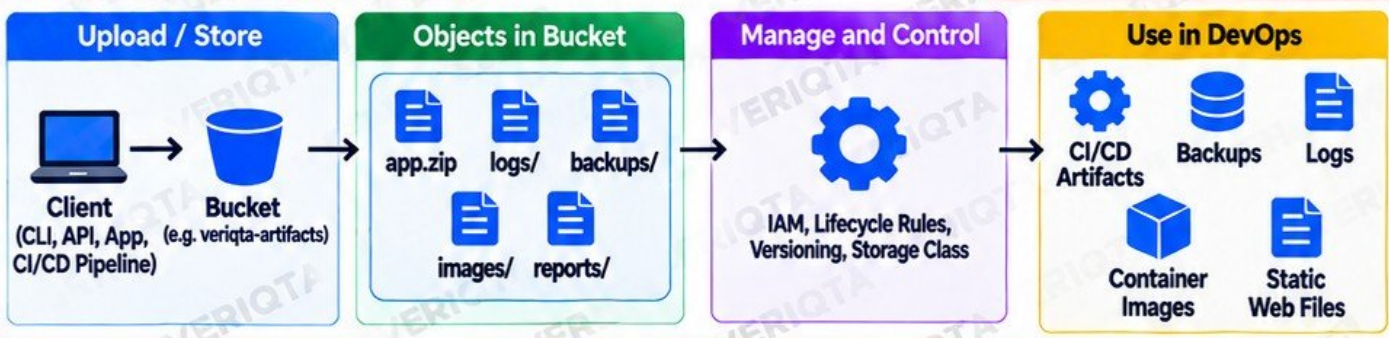
Storage Classes in GCP

Class	Use Case
Standard	Frequently accessed data (low latency).
Nearline	Infrequently accessed data (e.g. backups).
Coldline	Long-term data (e.g. disaster recovery).
Archive	Very long-term data (lowest cost).

Key Features

- ✓ High durability and availability.
- ✓ Integrates with IAM, CDN, BigQuery, Cloud Functions and more.
- ✓ Supports lifecycle rules and versioning.
- ✓ Scales automatically.

How Cloud Storage Works



Common DevOps Use Cases

-  Store build artifacts from CI/CD pipelines (e.g. binaries, Docker images, Helm charts).
-  Store application and system logs.
-  Keep backups and disaster recovery data.
-  Store static website files (e.g. frontend build).
-  Store configuration files and environment files.
-  Share large files between teams and services.

Example: Create a Bucket (gcloud CLI)

```

# Create a bucket in a specific region
gcloud storage buckets create gs://veriqa-artifacts \
  --location=us-central1

# Upload a file
gcloud storage cp app.zip gs://veriqa-artifacts/

# List objects
gcloud storage ls gs://veriqa-artifacts/

# Download a file
gcloud storage cp gs://veriqa-artifacts/app.zip .
  
```

Lifecycle Policy Example (JSON)

```

{
  "rule": [
    {
      "action": { "type": "SetStorageClass",
        "storageClass": "NEARLINE" },
      "condition": { "age": 30 }
    },
    {
      "action": { "type": "Delete" },
      "condition": { "age": 365 }
    }
  ]
}
  
```

Object Versioning

- ✓ Keeps multiple versions of each object.
- ✓ Helps recover from accidental deletion or overwrite.
- ✓ Can be enabled on a bucket.

Enable Versioning (CLI)

```

gcloud storage buckets update
gs://veriqa-artifacts \
  --versioning
  
```

IAM Access Examples

Role	Permissions
roles/storage.objectViewer	Read objects.
roles/storage.objectCreator	Upload new objects.
roles/storage.objectAdmin	Full control of objects.
roles/storage.admin	Full control of buckets and objects.

Key Takeaways

- ✓ Use buckets to store artifacts, logs, backups and more.
- ✓ Choose the right storage class for cost and access needs.
- ✓ Use IAM to control access.
- ✓ Enable lifecycle policies to manage data automatically.
- ✓ Use versioning to protect your data.
- ✓ Cloud Storage is an essential service for DevOps workflows.



veriqa



veriqa



veriqa



veriqa



veriqa



veriqa

09 Persistent Disk and VM Storage

What is Persistent Disk?

- Persistent Disk (PD) is Google Cloud's block storage for **Compute Engine VMs**.
- It provides **durable, high-performance storage** that remains even if the VM is stopped or deleted.
- Used for boot disks, data disks and applications that require persistent data.
- You can create snapshots to back up disks and restore them when needed.
- Persistent Disks are **zonal resources** and can be attached to VMs in the same zone.

Key Concepts

	Persistent Disk	Block storage for VMs. Data persists independently of the VM.
	Boot Disk	Disk that contains the operating system.
	Data Disk	Additional disk for application data.
	Snapshot	Point-in-time copy of a disk.
	Zone	Disks are zonal and must be in the same zone as the VM.
	Attach/Detach	Attach or detach disks from a running VM.
	Durability	Data is stored redundantly within a zone for high availability.

Disk Types and Use Cases

Type	Use Case	Key Features
Standard PD	General purpose workloads	Balanced performance and cost.
Balanced PD	Most workloads	Better performance than Standard.
SSD PD	High performance workloads	Low latency, high IOPS.
Extreme PD	Business critical applications	Highest performance and scalability.

Boot Disk vs Data Disk

Boot Disk	Data Disk
- Contains the OS. - Required to start the VM. - Typically not detached.	- Used for application data, databases, logs, etc. - Can be attached/detached. - Can have multiple disks per VM.

How Persistent Disk Works (Example Architecture)



Common Tasks (gcloud CLI)

```

# Create a persistent disk
gcloud compute disks create data-disk-1 \
  --size=100GB --type=pd-ssd --zone=us-central1-a

# Attach a disk to a VM
gcloud compute instances attach-disk dev-vm \
  --disk=data-disk-1 --zone=us-central1-a

# Create a snapshot
gcloud compute snapshots create data-disk-1-snap \
  --source-disk=data-disk-1 --zone=us-central1-a

# Create a disk from a snapshot
gcloud compute disks create restored-disk \
  --source-snapshot=data-disk-1-snap --zone=us-central1-a

# List disks
gcloud compute disks list --zone=us-central1-a
  
```

Attach Disk in Console

- 1 Go to Compute Engine > VM instances.
- 2 Select your VM.
- 3 Click Attach existing disk or Create new disk.
- 4 Choose the disk and click Attach.

Troubleshooting Storage Issues

- ✓ VM cannot see the disk: Check if it is attached and in the same zone.
- ✓ Disk full: Use `df -h` and clean up or increase size.
- ✓ Slow performance: Check disk type and IOPS.
- ✓ I/O errors: Check VM and disk health, review logs and metrics.
- ✓ Permission errors: Verify IAM roles.

Disk Lifecycle

	Create	Create a persistent disk in a specific zone.
	Attach	Attach the disk to a VM.
	Use	Read and write data on the disk.
	Snapshot	Create snapshots for backup.
	Restore	Create a new disk from a snapshot if needed.
	Delete	Delete the disk when you no longer need it (data is lost).

Useful Commands for Troubleshooting

Command	Purpose
<code>lsblk</code>	List block devices on the VM.
<code>df -h</code>	Check disk usage.
<code>sudo dmesg</code>	Check kernel messages for disk errors.
<code>gcloud compute disks describe DISK_NAME</code>	View disk details.
<code>gcloud compute snapshots list</code>	List snapshots.

Best Practices

- ✓ Use the right disk type for your workload.
- ✓ Take regular snapshots.
- ✓ Use separate data disks for databases and logs.
- ✓ Monitor disk usage, latency and IOPS.
- ✓ Use IAM to control who can create, attach or delete disks.
- ✓ Plan for backup and disaster recovery.

Key Takeaways

- ✓ Persistent Disk provides durable block storage for VMs.
- ✓ Boot disks contain the OS, data disks store application data.
- ✓ Disks are zonal and must be attached within the same zone.
- ✓ Snapshots help with backup and recovery.
- ✓ Use the right disk type based on performance and cost requirements.
- ✓ Monitor and troubleshoot storage issues to ensure application reliability.

10 Containers on GCP with Artifact Registry

What is Artifact Registry?

- Artifact Registry is Google Cloud's **managed container registry** for storing and managing container images and other artifacts (e.g. Helm charts, packages).
- It is the **recommended registry** for containers on GCP (replaces Container Registry).
- It integrates with IAM, VPC Service Controls, vulnerability scanning and Google Cloud services.

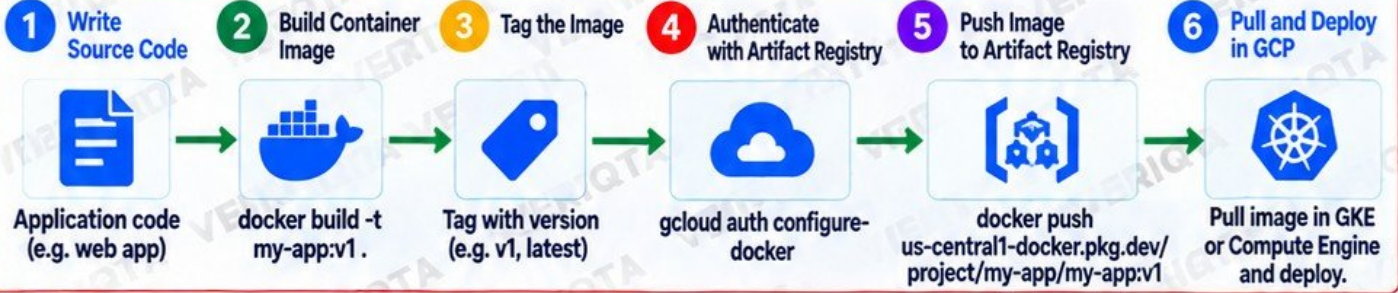
Key Concepts

	Container Image	A packaged application with its dependencies (e.g. Docker image).
	Tag	A label for an image version (e.g. v1, latest).
	Repository	Stores container images (e.g. my-app).
	Authentication	Use gcloud to authenticate Docker with Artifact Registry.
	Artifact Registry	Regional or multi-regional registry for containers and other artifacts.

Repository Types and Formats

Type	Description	Example
Docker	Stores container images.	us-central1-docker.pkg.dev/project/repo/image
Maven	Stores Java packages.	us-central1-maven.pkg.dev/project/repo
npm	Stores Node.js packages.	us-central1-ipm.pkg.dev/project/repo
Python	Stores Python packages.	us-central1-python.pkg.dev/project/repo
Apt / Yum	Stores OS packages.	us-central1-debian.pkg.dev/project/repo

End-to-End Flow: From Source Code to Deployable Artifact



Create a Repository (gcloud CLI)

```
# Create a Docker repository in a specific region
gcloud artifacts repositories create my-app \
  --repository-format=docker \
  --location=us-central1 \
  --description="Repository for my application images"
```

Authenticate Docker with Artifact Registry

```
gcloud auth configure-docker us-central1-docker.pkg.dev
```

Build, Tag and Push an Image

```
# Build the image
docker build -t my-app:v1 .

# Tag the image
docker tag my-app:v1 us-central1-docker.pkg.dev/PROJECT_ID/my-app/my-app:v1

# Push the image
docker push us-central1-docker.pkg.dev/PROJECT_ID/my-app/my-app:v1
```

List Images in the Repository

```
gcloud artifacts docker images list \
  us-central1-docker.pkg.dev/PROJECT_ID/my-app
```

Pull and Run the Image

```
# Authenticate (if not already)
gcloud auth configure-docker us-central1-docker.pkg.dev

# Pull the image
docker pull us-central1-docker.pkg.dev/PROJECT_ID/my-app/my-app:v1

# Run locally
docker run -p 8080:80 my-app:v1
```

IAM Roles for Artifact Registry

Role	Permissions
roles/artifactregistry.reader	Read images from repositories.
roles/artifactregistry.writer	Push and pull images.
roles/artifactregistry.admin	Full control of repositories.
roles/artifactregistry.createOnPush	Allows creating repositories on image push.

Common DevOps Use Cases

- ✓ Store container images for CI/CD pipelines.
- ✓ Keep multiple versions of application images.
- ✓ Use image tags for environment deployments (dev, test, prod).
- ✓ Integrate with GKE, Cloud Run or Compute Engine.
- ✓ Enable vulnerability scanning for images.
- ✓ Store other artifacts (Helm charts, packages).

Key Takeaways

- ✓ Artifact Registry is the standard container registry on GCP.
- ✓ Build, tag and push images using Docker and gcloud.
- ✓ Use IAM to control access.
- ✓ Keep images versioned with tags.
- ✓ Integrates with GKE, Cloud Run and other GCP services.
- ✓ Supports vulnerability scanning and multiple artifact formats.

What is GKE?

- Google Kubernetes Engine (GKE) is Google Cloud's **managed Kubernetes service**.
- It runs Kubernetes clusters on GCP infrastructure, so you don't have to manage the **control plane**.
- You deploy, manage and scale **containerized applications** using Kubernetes, while Google manages the underlying infrastructure.
- GKE integrates with other **GCP services** such as VPC, Cloud IAM, Cloud Load Balancing, Cloud Logging and Artifact Registry.

Key Kubernetes Concepts

Component	Description
 Cluster	A set of nodes that run containerized applications.
 Node	A VM in GCP (Compute Engine) that runs Pods.
 Pod	The smallest deployable unit. Runs one or more containers.
 Deployment	Manages Pods and ensures the desired number of replicas are running.
 Service	Provides stable network access to Pods (ClusterIP, NodePort, LoadBalancer).
 Namespace	Logical isolation for resources within a cluster.

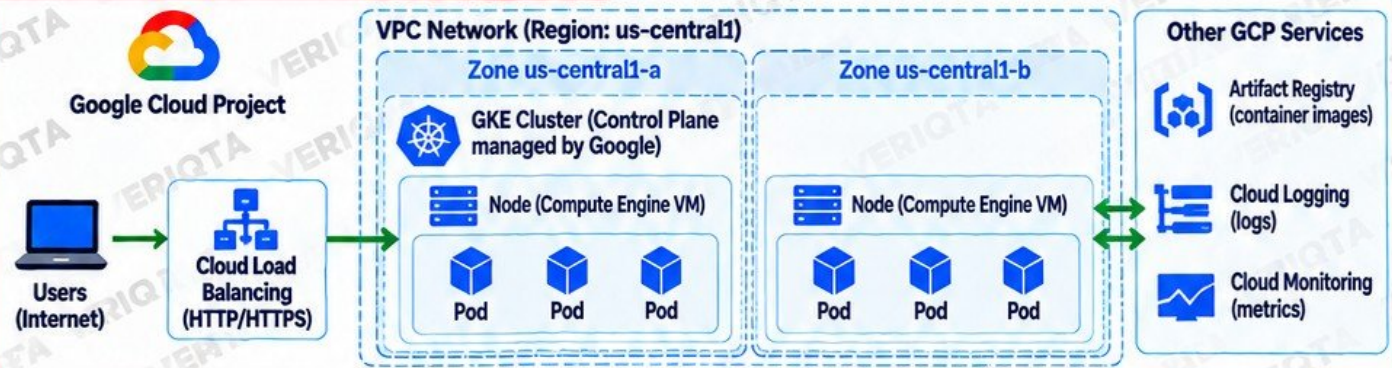
GKE Cluster Types

Type	Description
Standard Cluster	You manage the nodes. More control, custom configurations.
Autopilot Cluster	Google manages the nodes. Simpler, scales automatically, lower operational overhead.

GKE Integrates With

	VPC (networking)
	Cloud IAM (access control)
	Cloud Load Balancing (traffic to applications)
	Artifact Registry (container images)
	Cloud Logging (logs)
	Cloud Monitoring (metrics)

How GKE Maps to GCP Infrastructure



Basic Deployment Flow

- Build container image and push to Artifact Registry.
- Create or connect to a GKE cluster.
- Deploy application using Kubernetes manifests (Deployment, Service, etc.).
- Kubernetes creates Pods on cluster nodes.
- Service exposes the application (internal or external).
- Traffic reaches the application through a Service (and optionally a Load Balancer).

Common DevOps Use Cases

- ✓ Deploying containerized applications.
- ✓ CI/CD pipelines that build and deploy to GKE.
- ✓ Scaling applications automatically.
- ✓ Blue/green and rolling updates.
- ✓ Running microservices.
- ✓ Using ConfigMaps and Secrets for configuration.
- ✓ Integrating with Cloud Load Balancing for external access.

Example: Simple Deployment (YAML)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
          image: us-central1-docker.pkg.dev/my-project/my-repo/my-app:v1
          ports:
            - containerPort: 8080
```

Key Takeaways

- ✓ GKE is a managed Kubernetes service on GCP.
- ✓ A cluster runs on Compute Engine VMs (nodes).
- ✓ You deploy applications using Deployments and Services.
- ✓ Use Artifact Registry to store container images.
- ✓ Services provide stable access to your applications.
- ✓ GKE integrates with networking, IAM, monitoring and other GCP services.
- ✓ Start with simple deployments and grow to advanced configurations.

Access the Cluster (gcloud CLI)

```
# Create a GKE cluster
gcloud container clusters create my-cluster \
  --region us-central1 \
  --num-nodes 3 \
  --machine-type e2-standard-4

# Get credentials to use kubectl
gcloud container clusters get-credentials \
  my-cluster --region us-central1

# Check nodes
kubectl get nodes
```

Service Types

Type	Use Case	Access
ClusterIP	Internal access within cluster	Not accessible from internet
NodePort	Exposes a port on each node	Accessible via node IP and port
LoadBalancer	Creates a GCP load balancer	Accessible from internet (external IP)
ExternalName	Maps to an external DNS name	Accesses external service



veriQTA



veriQTA



veriQTA



veriQTA



veriQTA

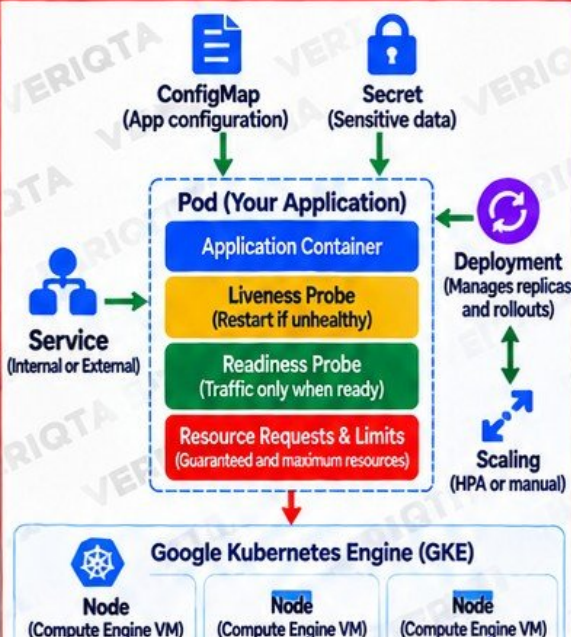


veriQTA

Key Concepts

	ConfigMap	Store non-sensitive configuration data.
	Secret	Store sensitive data (e.g. API keys, passwords).
	Probes	Health checks to keep applications healthy.
	Resources	CPU and memory requests and limits.
	Scaling	Scale manually or automatically.
	Rollouts	Update applications with zero or minimal downtime.
	kubectl	First-line tool for troubleshooting.

How These Components Work Together



Common kubectl Commands

Task	Command
View pods	<code>kubectl get pods</code>
View services	<code>kubectl get svc</code>
View deployments	<code>kubectl get deployments</code>
View logs	<code>kubectl logs <pod-name></code>
Describe a resource	<code>kubectl describe pod <pod-name></code>
Check events	<code>kubectl get events</code>
Execute a command	<code>kubectl exec -it <pod-name> -- /bin/sh</code>
View ConfigMap	<code>kubectl get configmap <name> -o yaml</code>
View Secret	<code>kubectl get secret <name> -o yaml</code>
Rollout status	<code>kubectl rollout status deployment/<name></code>
Roll back	<code>kubectl rollout undo deployment/<name></code>
Scale manually	<code>kubectl scale deployment <name> --replicas=3</code>

ConfigMap Example (YAML)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: default
data:
  APP_ENV: "production"
  LOG_LEVEL: "info"
  API_URL: "https://api.example.com"
```

Secret Example (YAML)

```
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
  namespace: default
type: Opaque
data:
  DB_PASSWORD: <base64-encoded-value>
  API_KEY: <base64-encoded-value>
```

Probes Example (YAML)

```
livenessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5
```

Resource Requests and Limits (YAML)

```
resources:
  requests:
    cpu: "250m" # 0.25 CPU
    memory: "256Mi" # 256 MiB
  limits:
    cpu: "500m" # 0.5 CPU
    memory: "512Mi" # 512 MiB
```

Scaling Options

Manual Scaling

```
kubectl scale deployment my-app \
  --replicas=5
```

Horizontal Pod Autoscaler (HPA)

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Rollout and Update

```
# Update image
kubectl set image deployment/my-app \
  my-app=us-central1-docker.pkg.dev/
  project/my-app:v2

# Check rollout status
kubectl rollout status deployment/my-app

# Roll back if needed
kubectl rollout undo deployment/my-app
```

Common Troubleshooting Steps

- 1 Check pod status and events:
`kubectl get pods, kubectl describe pod`
- 2 Check logs:
`kubectl logs <pod-name>`
- 3 Check resource usage:
`kubectl top pods (requires metrics-server)`
- 4 Verify ConfigMaps and Secrets:
`kubectl get configmap, kubectl get secret`
- 5 Check probes and container health:
`kubectl describe pod <pod-name>`
- 6 Look at deployment and rollout status:
`kubectl get deployments, kubectl rollout status`

Key Takeaways

- ✓ Use ConfigMaps for configuration and Secrets for sensitive data.
- ✓ Configure liveness and readiness probes.
- ✓ Set appropriate resource requests and limits.
- ✓ Scale manually or use HPA for automatic scaling.
- ✓ Use rolling updates for zero or minimal downtime.
- ✓ kubectl is your first tool for troubleshooting.
- ✓ Monitor logs, events and metrics to find issues quickly.



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

13 Serverless Deployment with Cloud Run

What is Cloud Run?

- Cloud Run is a **fully managed serverless platform** on Google Cloud for running containerized applications.
- You deploy a container image, and Google manages the underlying infrastructure.
- It automatically scales based on incoming traffic, including **scale to zero**.
- You only pay for the resources used by your application.
- It is ideal for **stateless services**, APIs, web applications and background workers.

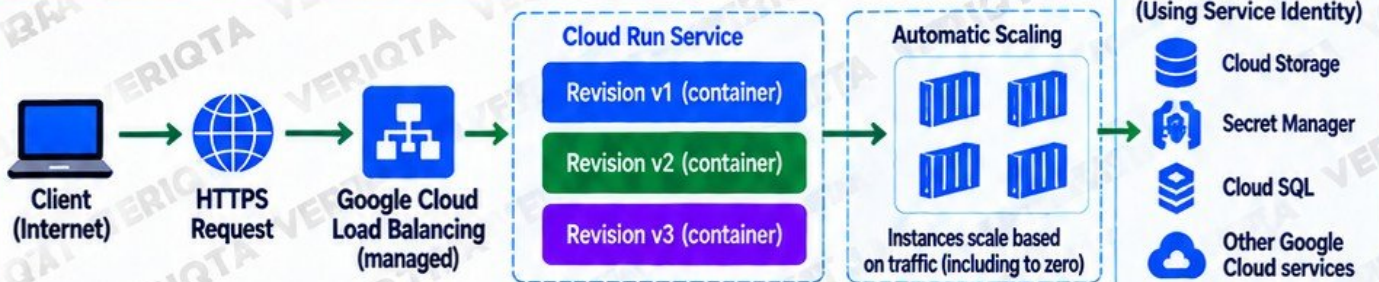
Key Concepts

	Container	Your application packaged as a container image.
	Service	A long-running endpoint for your application.
	Revision	Immutable version of your container. New revision is created for each change.
	Traffic	Control how traffic is distributed between revisions.
	Scaling	Automatically scales up and down, including to zero.
	Service Identity	Uses a service account to access other GCP services.
	Environment Configuration	Set environment variables, secrets and other settings.

Cloud Run vs GKE

Feature	Cloud Run	GKE
Management	Fully managed (serverless)	You manage the cluster
Scaling	Automatic (includes zero)	Automatic or manual (nodes)
Infrastructure	No servers to manage	You manage nodes and cluster
Use Case	Web apps, APIs, background jobs	Complex apps, stateful workloads
Networking	Built-in, simple	More control and configuration
Cost	Pay per use (requests + runtime)	Pay for cluster nodes (even if idle)

How Cloud Run Works (Architecture)



Deploy to Cloud Run (gcloud CLI)

```
# Build and push container image to Artifact Registry
gcloud builds submit --tag \
  us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1 \

# Deploy to Cloud Run
gcloud run deploy my-app \
  --image us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1 \
  --region us-central1 \
  --allow-unauthenticated

# Update with a new image (creates a new revision)
gcloud run deploy my-app \
  --image us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v2 \
  --region us-central1
```

Scaling Behavior

- ✓ Scales automatically based on incoming traffic.
- ✓ Can scale to zero when there is no traffic.
- ✓ You can set a minimum and maximum number of instances.
- ✓ Handles traffic spikes without manual intervention.
- ✓ You only pay for the time your containers are running and the requests.

Example: Cloud Run Service (YAML)

```
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: my-app
spec:
  template:
    metadata:
      annotations:
        autoscaling.knative.dev/minScale: "0"
        autoscaling.knative.dev/maxScale: "10"
    spec:
      serviceAccountName: my-service-account
      containers:
        - image: us-central1-docker.pkg.dev/my-proj/my-repo/my-app:v1
      env:
        - name: APP_ENV
          value: "production"
        - name: API_KEY
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: api-key
```

Traffic Management

- Each deployment creates a new revision.
- You can split traffic between revisions (e.g. 90/10).
- Use this for canary releases, A/B testing and rollbacks.
- Traffic can be adjusted using gcloud or the console.

Environment Configuration

- Environment Variables (non-sensitive)
- Secrets (sensitive data)
- Other settings (concurrency, CPU, memory, timeout, etc.)

Key Takeaways

- ✓ Cloud Run lets you run containers without managing servers.
- ✓ Each change creates a new revision.
- ✓ It automatically scales, including to zero.
- ✓ Use service identity to access other GCP services.
- ✓ Configure environment variables and secrets properly.
- ✓ Use traffic management for safe rollouts.
- ✓ Choose Cloud Run for simpler, stateless workloads. Use GKE for complex or stateful applications.

Common Use Cases



First-Line Troubleshooting (kubectl alternatives)

- ✓ Check service status: `gcloud run services describe my-app`
- ✓ View revisions: `gcloud run revisions list --service my-app`
- ✓ Check logs: `gcloud logging read "resource.type=cloud_run_revision"`
- ✓ Check traffic configuration: `gcloud run services describe my-app --format="value(status.traffic)"`
- ✓ Check environment variables and settings.
- ✓ Verify service account permissions.
- ✓ Check error messages and revision logs in Cloud Logging.

14 Infrastructure as Code with Terraform

What is Terraform?

- Terraform is an **open source Infrastructure as Code (IaC)** tool by HashiCorp.
- You define your infrastructure in code and Terraform provisions, updates and manages it.
- It works with multiple cloud providers, including **Google Cloud** using the Google provider.
- Terraform helps you create repeatable, version-controlled and consistent infrastructure.

Key Concepts

	Provider	Plugin that lets Terraform talk to a cloud platform (e.g. Google Cloud).
	Resource	A component of your infrastructure (e.g. VM, VPC, bucket).
	Variable	Input values to make your configuration flexible and reusable.
	Output	Values returned after creation (e.g. VM IP).
	State	File that keeps track of the real infrastructure (i.e. what Terraform created).
	Plan	Shows what changes will be made.
	Apply	Creates or updates the infrastructure.

Google Cloud Provider



Uses the **google** provider to manage GCP resources.

```
terraform {
  required_providers {
    google = {
      source = "hashicorp/google"
      version = "~> 5.0"
    }
  }
}

provider "google" {
  project = "my-gcp-project"
  region = "us-central1"
}
```

How Terraform Works (Workflow)



Example: Create a Google Compute Engine VM

```
# variables.tf
variable "project_id" {
  description = "GCP Project ID"
  type        = string
}

# main.tf
resource "google_compute_instance" "vm" {
  name         = "my-vm"
  machine_type = "e2-micro"
  zone         = "us-central1-a"
  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-12"
    }
  }
}

# outputs.tf
output "vm_name" {
  value = google_compute_instance.vm.name
}

output "vm_internal_ip" {
  value = google_compute_instance.vm.network_interface[0].network_ip
}
```

State Management

- ✓ Terraform stores the state file (**terraform.tfstate**) which tracks the real infrastructure.
- ✓ Keep the state file in a secure remote backend (e.g. Google Cloud Storage) for team use.
- ✓ Never delete or manually edit the state file.
- ✓ Use state locking to prevent concurrent changes.

Plan vs Apply

Command	Purpose
terraform init	Download and initialize providers.
terraform plan	Show what changes will be made.
terraform apply	Create or update the infrastructure.
terraform destroy	Remove the infrastructure.

Useful Commands

```
terraform init # Initialize Terraform
terraform plan # Preview changes
terraform apply # Create/update resources
terraform destroy # Remove resources
terraform validate # Check configuration
terraform fmt # Format code
terraform show # Show current state
terraform output # View output values
```

Common GCP Resources

-  google_compute_instance VM instances
-  google_compute_network VPC networks
-  google_compute_subnetwork Subnets
-  google_storage_bucket Cloud Storage buckets
-  google_container_cluster GKE clusters
-  google_cloud_run_service Cloud Run services
-  google_dns_managed_zone Cloud DNS zones

Key Takeaways

- ✓ Terraform lets you safely provision GCP infrastructure through code.
- ✓ Use the Google provider to manage GCP resources.
- ✓ Understand providers, resources, variables, outputs, state, plan and apply.
- ✓ Keep your state file secure and use best practices.
- ✓ Terraform helps you build consistent, repeatable and version-controlled infrastructure.

Best Practices

- ✓ Use variables and outputs.
- ✓ Keep state file in a remote backend (e.g. GCS).
- ✓ Use version control (Git).
- ✓ Use workspaces for multiple environments (dev, staging, prod).
- ✓ Follow the principle of least privilege with IAM.
- ✓ Always review the plan before applying.
- ✓ Use modules for reusable infrastructure.
- ✓ Tag your resources.

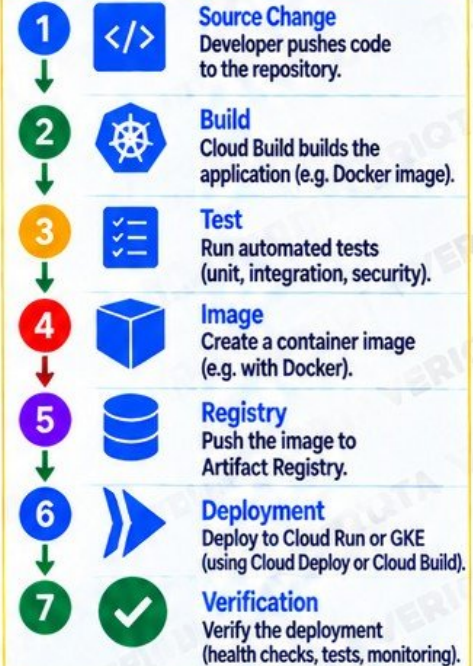
What is CI/CD on Google Cloud?

- CI/CD (Continuous Integration and Continuous Delivery) automatically builds, tests and deploys your application when there are code changes.
- Google Cloud uses **Cloud Build** and other services to implement CI/CD pipelines.
- It helps you deliver changes **faster, safer and more consistently**.
- You push code, the pipeline builds a container image, runs tests, stores the image in **Artifact Registry**, deploys it (e.g. to **Cloud Run** or **GKE**) and verifies the deployment.

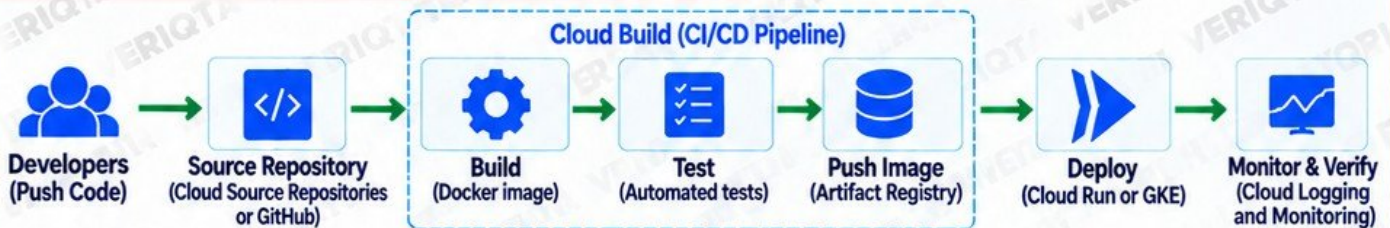
Key Services

	Cloud Build	Builds and runs your CI/CD pipeline.
	Source Repository (or GitHub)	Stores your source code.
	Artifact Registry	Stores container images and other artifacts.
	Cloud Run / GKE	Deploys your application.
	Cloud Logging	Stores build and deployment logs.
	Cloud Monitoring	Monitors your application and pipeline.

Typical CI/CD Pipeline Flow



End-to-End Architecture on Google Cloud



Example: cloudbuild.yaml

steps:

```

- name: 'gcr.io/cloud-builders/docker'
  args: ['build', '-t', 'us-central1-docker.pkg.dev/$PROJECT_ID/my-app/my-app:$SHORT_SHA', '.']
- name: 'gcr.io/cloud-builders/docker'
  args: ['push', 'us-central1-docker.pkg.dev/$PROJECT_ID/my-app/my-app:$SHORT_SHA']
- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
  args: ['gcloud', 'run', 'deploy', 'my-app', '--image', 'us-central1-docker.pkg.dev/$PROJECT_ID/my-app/my-app:$SHORT_SHA', '--region', 'us-central1', '--allow-unauthenticated']
  
```

Pipeline in Action (Real Flow)

- Developer pushes code to the main branch.
- Cloud Build is triggered automatically.
- It builds a Docker image.
- Runs your tests.
- Pushes the image to Artifact Registry.
- Deploys the image to Cloud Run (or GKE).
- Runs post-deployment checks (e.g. health endpoint).

Deployment Options

Deploy to Cloud Run (Serverless)



- Simple and fully managed.
- Scales automatically to zero.
- Great for containers, APIs and web apps.

Deploy to GKE (Kubernetes)



- Use for more complex applications.
- Full Kubernetes features.
- Better for microservices and advanced workloads.

Useful gcloud Commands

```

# Trigger a build manually
gcloud builds submit --config=cloudbuild.yaml .

# View build history
gcloud builds list

# Deploy to Cloud Run
gcloud run deploy my-app --image \
  us-central1-docker.pkg.dev/$PROJECT_ID/my-app/my-app:latest \
  --region us-central1

# View logs
gcloud logging read "resource.type=cloud_run_revision"
  
```



Key Takeaways

- CI/CD on Google Cloud is powered by Cloud Build.
- It automatically builds, tests, stores and deploys your application.
- Use Artifact Registry to store container images.
- Deploy to Cloud Run for simple workloads or GKE for complex applications.
- Add automated tests and verification steps.
- Monitor builds and deployments using Cloud Logging and Cloud Monitoring.
- It helps you deliver changes quickly and safely.

Common Use Cases



Web applications



APIs and microservices



Mobile backend services



Batch and scheduled jobs



Infrastructure automation



Event-driven applications



veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

Why Secrets Should Not Live in Git

- ✗ Git repositories are often public or accessible to many people.
- ✗ Secrets can be leaked and used by attackers.
- ✗ Even deleted secrets can still be found in Git history.
- ✗ Hardcoded credentials make rotation difficult.
- ✗ It violates security best practices and compliance requirements.

What is Secret Manager?

Secret Manager is a fully managed service on Google Cloud for storing and accessing sensitive data such as API keys, passwords, database credentials and tokens.

-  **Store**
Store sensitive data as secrets.
-  **Manage Versions**
Keep multiple versions of each secret.
-  **Control Access**
Use IAM to control who can access secrets.
-  **Rotate**
Easily rotate secrets when needed.
-  **Integrate**
Use with Compute Engine, GKE, Cloud Run and other GCP services.

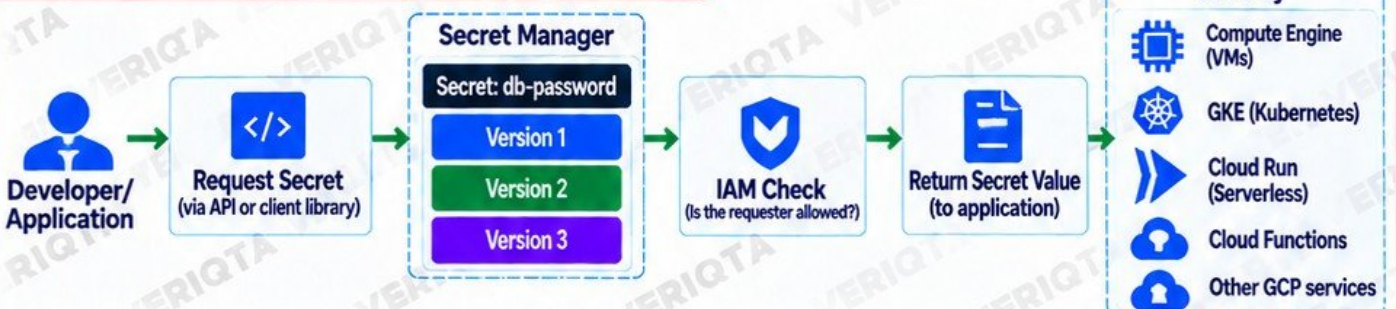
Common Types of Secrets

-  **API keys**
(e.g. Google APIs, third-party services)
-  **Database credentials**
(username and password)
-  **Service account keys**
(when required)
-  **Tokens**
(e.g. OAuth, JWT, access tokens)
-  **SMTP credentials**
-  **Any sensitive configuration**
(e.g. webhook URLs, private keys)

IAM-Controlled Access

-  Use IAM roles to allow only specific users, groups or service accounts to access secrets.
-  Follow the principle of least privilege.
-  You can control access at the project, folder or organization level.

How Secret Manager Works (Architecture)



Example: Store a Secret (gcloud CLI)

```
# Create a secret
gcloud secrets create db-password \
  --replication-policy=automatic

# Add a secret value
echo -n "my-db-password" | gcloud secrets \
  versions add db-password --data-file=-
```

Example: Access a Secret

gcloud CLI

```
# Get the latest secret value
gcloud secrets versions access \
  latest --secret=db-password
```

Application Code (Go)

```
import (
    "cloud.google.com/go/secretmanager/apiv1"
)
// Use client to access secret
// returns the secret value
```

Using Secrets in GKE (Workload Identity)



Secret Rotation

-  Add a new secret version.
-  Update applications to use the new version.
-  Test to ensure the new version works.
-  Remove or disable the old version.
-  Automate rotation using Cloud Scheduler or your CI/CD pipeline.

Best Practices

-  Never store secrets in Git repositories.
-  Use Secret Manager for all sensitive data.
-  Use IAM and workload identity instead of long-lived keys.
-  Rotate secrets regularly.
-  Monitor access using Cloud Logging and audit logs.
-  Use environment variables or mounted files to access secrets.
-  Follow the principle of least privilege.

Common Use Cases

-  Database credentials for applications
-  API keys for external services
-  Secrets for Kubernetes workloads
-  Service account keys (when needed)
-  Configuration values (e.g. webhooks)
-  TLS certificates and private keys

Key Takeaways

-  Secret Manager keeps your sensitive data secure.
-  Secrets should never be hardcoded or stored in Git.
-  Use IAM to control who can access secrets.
-  Workload identity allows secure access from GKE.
-  Rotate secrets regularly.
-  Integrate with your applications using environment variables or client libraries.
-  Follow security best practices to protect your systems.

What is Observability?

- Observability helps you understand what is happening in your applications and infrastructure.
- Google Cloud provides **Cloud Monitoring** for metrics, dashboards and alerts, and **Cloud Logging** for logs.
- It gives you the data you need to detect issues, investigate problems and keep your services reliable.
- Instead of guessing, you use real signals from your systems.

Key Concepts

	Metrics	Numerical data over time (e.g. CPU, memory, latency, request count).
	Logs	Timestamped events and messages from applications and infrastructure.
	Dashboards	Visualize key metrics and logs for easy monitoring.
	Alerts	Notify you when something is wrong (e.g. high error rate, CPU spike).
	Traces (Optional)	Follow a request through multiple services to find where it fails.
	Uptime Checks	Periodically check if your service is available.

Google Cloud Services

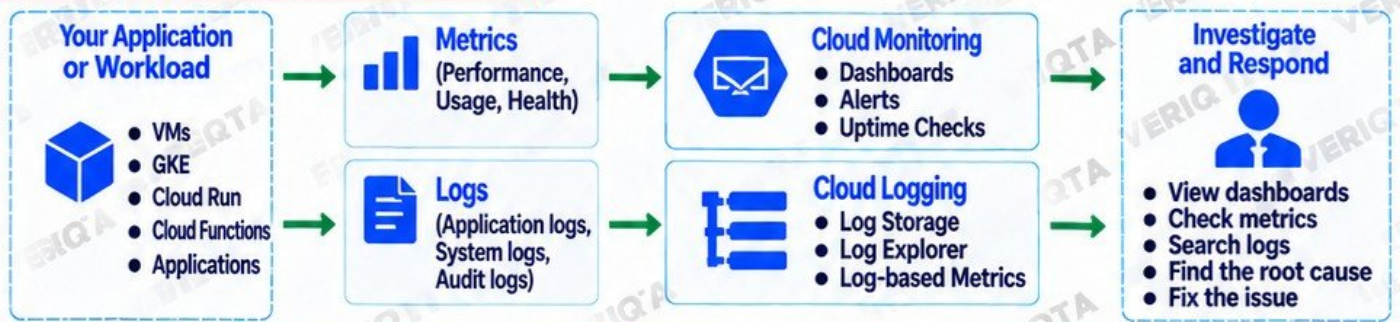
Cloud Monitoring (Metrics, Dashboards, Alerts)

- Collects metrics from GCP services, VMs, GKE, Cloud Run and your applications.
- Create custom dashboards.
- Set up alerting policies.

Cloud Logging (Logs and Audit Logs)

- Collects and stores logs from GCP services, applications and infrastructure.
- Use Logs Explorer to search and filter logs.
- Includes audit logs for security and compliance.

How It Works (Architecture)



Example: View Metrics (gcloud CLI)

```
# List available metrics
gcloud monitoring metrics list --filter="metric.type
~'compute.googleapis.com'"

# View CPU utilization for a VM
gcloud monitoring time-series list \
--filter="metric.type='compute.googleapis.com/
instance/cpu/utilization'" \
--interval="end-time=now;start-time=PT1H" \
--limit=5
```

Example: Query Logs (Logs Explorer)

```
# Find error logs for a service
resource.type="k8s_container"
resource.labels.cluster_name="my-cluster"
severity>=ERROR
jsonPayload.message:"database"

# Filter logs for a specific time range
timestamp >= "2024-01-01T00:00:00Z"
timestamp <= "2024-01-01T01:00:00Z"
```

Example: Create an Alert

```
# Create an alerting policy for high CPU
gcloud alpha monitoring policies create \
--policy-from-file=cpu-alert.yaml
```

Alert Condition (YAML)

```
displayName: "High CPU Usage"
conditionThreshold:
  filter: metric.type="compute.googleapis.com/
instance/cpu/utilization"
  comparison: COMPARISON_GT
  thresholdValue: 0.8
  duration: 300s
```

Common Metrics to Monitor

-  CPU and memory utilization
-  Request count and latency
-  Error rate (4xx, 5xx)
-  Disk and network usage
-  Application specific metrics (e.g. queue length, active connections)

Common Logs to Check

-  Application logs (errors, exceptions)
-  Container logs (GKE, Cloud Run)
-  System logs (VMs)
-  Audit logs (security and access)
-  Load balancer logs
-  Database logs (e.g. Cloud SQL)

Troubleshooting Steps

- 1 Check the dashboard for overall impact.
- 2 Look at metrics (CPU, memory, latency, error rate).
- 3 Search logs for error messages and patterns.
- 4 Correlate the time of the issue across metrics and logs.
- 5 Use traces (if available) to follow a request.
- 6 Identify the root cause and apply a fix.

Key Takeaways

- ✓ Use Cloud Monitoring and Cloud Logging to understand your systems.
- ✓ Create dashboards and alerts for key metrics.
- ✓ Centralize logs and use Logs Explorer to investigate issues.
- ✓ Monitor both infrastructure and application signals.
- ✓ Set up alerts before customers are affected.
- ✓ Use real data, not guesswork.

Key Concepts

	Zone	A separate data center within a region.
	Region	A geographical area with multiple zones.
	Redundancy	Running resources in multiple zones or regions to avoid a single point of failure.
	Managed Instance Group (MIG)	Automatically creates and manages a group of identical VMs.
	Autoscaling	Adjusts the number of instances based on demand.
	Health Check	Monitors instance health and replaces unhealthy instances.
	Failure Domain	A component or area that can fail independently (e.g. instance, zone, region).

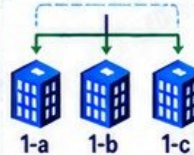
Zones vs Regions

Zone	Region
- Single data center within a region. - Lower latency (within the zone). - Used for high availability within a region.	- Multiple zones (typically 3 or more). - Provides higher resilience. - Used for disaster recovery and regional redundancy.

Zone
e.g. us-central1-a



Region
e.g. us-central1 (3 zones)



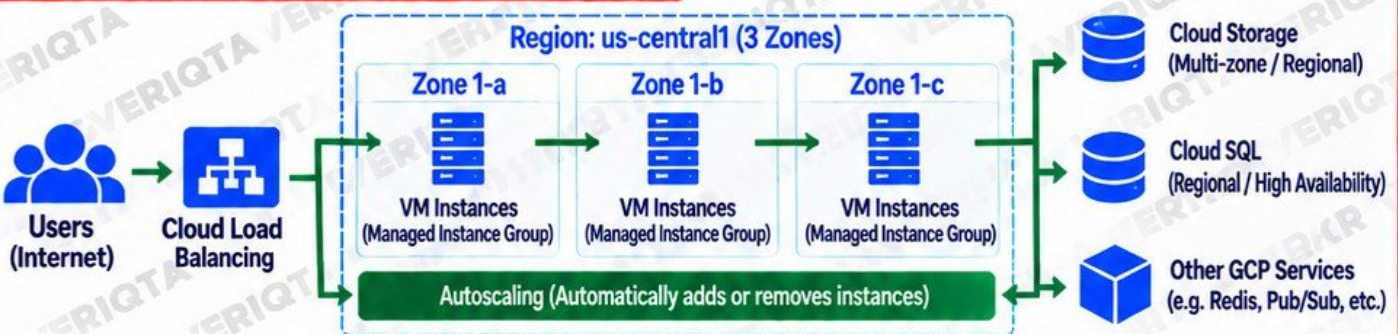
Why It Matters

- ✓ Zones and regions protect you from data center and regional failures.
- ✓ Redundancy keeps your applications available when components fail.
- ✓ Autoscaling helps you handle traffic spikes and reduces costs.
- ✓ Health checks automatically replace unhealthy instances.
- ✓ Designing for failure ensures your system continues to operate even when parts of it fail.

Common Failure Scenarios

- ✗ A VM becomes unhealthy.
- ✗ An entire zone goes down.
- ✗ A region experiences an outage.
- ✗ A load balancer target fails.
- ✗ Traffic suddenly increases.
- ✗ A dependency service becomes unavailable.

High Availability Architecture on GCP



Managed Instance Group (MIG)

- ✓ Creates and manages a group of identical VMs.
- ✓ Automatically replaces unhealthy instances.
- ✓ Works with autoscaling and health checks.
- ✓ Distributes instances across multiple zones.
- ✓ Ideal for stateless applications.

```
# Create a managed instance group
gcloud compute instance-groups managed create my-mig \
  --template=my-template \
  --zone=us-central1-a

# Set autoscaling policy
gcloud compute instance-groups managed set-autoscaling \
  my-mig --zone=us-central1-a \
  --min-num-replicas=2 --max-num-replicas=10 \
  --target-cpu-utilization=0.6
```

Autoscaling

- ✓ Increases instances when demand rises.
- ✓ Decreases instances when demand drops.
- ✓ Can be based on CPU, memory, or custom metrics (e.g. request count).
- ✓ Helps maintain performance and optimize costs.

Autoscaling Based on CPU Utilization



Health Checks

- ✓ Periodically check if instances are healthy.
- ✓ Used by load balancers and MIGs.
- ✓ Automatically remove and replace unhealthy instances.
- ✓ Can check HTTP, HTTPS, or TCP.
- ✓ Prevent traffic from going to failed instances.

Designing for Component Failure

- ✓ Assume that any component can fail (instance, zone, service).
- ✓ Use multiple zones and regions for redundancy.
- ✓ Keep your application stateless where possible.
- ✓ Use managed services (e.g. Cloud SQL, Pub/Sub) with high availability.
- ✓ Implement proper health checks, autoscaling and monitoring.
- ✓ Test failure scenarios regularly (e.g. zone outage).

Best Practices

- ✓ Deploy across multiple zones.
- ✓ Use regional or multi-zone managed services.
- ✓ Enable autoscaling with appropriate limits.
- ✓ Use health checks for all critical services.
- ✓ Design for failure, not just for normal operation.
- ✓ Keep data backups and disaster recovery plans.
- ✓ Monitor and alert on key metrics.

 Key Takeaways

- ✓ Zones provide high availability within a region.
- ✓ Regions provide higher resilience and protect against larger failures.
- ✓ Managed instance groups, autoscaling and health checks keep your applications running.
- ✓ Designing for failure ensures your system remains available even when components fail.

Common Symptoms

- ❗ Application is not accessible or returns errors.
- ❗ High latency or timeouts.
- ❗ Some users can access but others cannot.
- ❗ Services are unhealthy or crashing.
- ❗ Recent deployment or configuration change.
- ❗ Dependencies (database, API) not responding.

Incident Troubleshooting Flow

- 1  Confirm the issue
What is affected? Is it all users or a specific region, service or endpoint?
- 2  Check IAM and access
Are you authenticated and authorized to access the resources?
- 3  Check DNS
Does the domain resolve to the correct IP or load balancer?
- 4  Check networking and firewall rules
Is traffic allowed to reach your application?
- 5  Check VM or container state
Are the instances/pods running and healthy?
- 6  Check application logs
Look for errors and exceptions.
- 7  Check health checks
Are load balancer and service health checks passing?
- 8  Check dependencies
Are databases, APIs and external services reachable and healthy?
- 9  Review recent changes
Look at recent deployments, configuration changes or infrastructure updates.

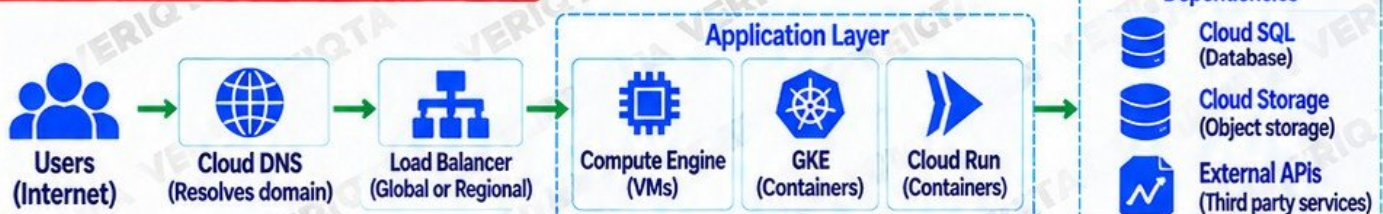
Key GCP Services to Check

-  IAM (Identity and Access Management)
-  Cloud DNS (Domain name resolution)
-  VPC and Firewall Rules (Networking and traffic control)
-  Compute Engine / GKE / Cloud Run (Where your application runs)
-  Cloud Logging (Application and system logs)
-  Cloud Monitoring (Metrics, health checks, alerts)
-  Cloud SQL / Cloud Spanner (Managed databases)
-  Other Dependencies (APIs, external services, third party)

Common Root Causes

- ❌ Missing or incorrect IAM permissions.
- ❌ DNS misconfiguration or stale records.
- ❌ Firewall rules blocking traffic.
- ❌ VM/pod not running or unhealthy.
- ❌ Application error or misconfiguration.
- ❌ Failed health checks.
- ❌ Database or external API outage.
- ❌ Issues after a recent change.

Architecture View (End-to-End)



Useful gcloud Commands

```
# Check active account
gcloud auth list
# Check project
gcloud config get-value project
# Check VM instances
gcloud compute instances list
# Check GKE clusters and pods
gcloud container clusters list
kubectl get pods -A
# Check firewall rules
gcloud compute firewall-rules list
# Check load balancer health checks
gcloud compute health-checks list
# Check recent operations
gcloud operations list --limit=10
```

Health Check Validation

- ✓ Check load balancer health check status.
- ✓ Verify instance/pod readiness and liveness.
- ✓ Confirm the health check path and port.
- ✓ Ensure the application returns the expected response.
- ✓ Check Cloud Monitoring for failing health checks.

Log Investigation (Cloud Logging)

```
# View recent error logs
gcloud logging read \
  'resource.type="k8s_container" \
  severity>=ERROR' \
  --limit=50

# Filter logs for a specific service
gcloud logging read \
  'resource.labels.service_name="my-app"' \
  --limit=50 --format=json
```

Recent Changes to Review

- 🔴 Recent deployments (Cloud Build, manual).
- 🔴 Configuration changes (env variables, config maps).
- 🔴 Infrastructure changes (firewall rules, IAM, network).
- 🔴 New releases or version upgrades.
- 🔴 Changes to dependencies (database, external APIs).
- 🔴 Look at Cloud Audit Logs and deployment history.

DNS and Network Checks

```
# Check DNS resolution
nslookup your-domain.com
# Test connectivity
ping <internal-ip>
# Test HTTP/HTTPS
curl -I https://your-domain.com

# Check firewall rules
gcloud compute firewall-rules list \
  --filter="network=<<vpc-name>"
```

Key Takeaways

- 💡 Follow a structured troubleshooting flow.
- ✓ Check each layer: IAM, DNS, network, compute/container, application, dependencies.
- ✓ Use real logs and metrics, not guesswork.
- ✓ Compare with the last known good state.
- ✓ Recent changes are a common cause.
- ✓ Design for observability to make troubleshooting easier.

20 Production Deployment, End to End

What You Will Build

- ✓ A complete, real-world production deployment flow on Google Cloud.
- ✓ From code commit to a running application with monitoring, alerts and rollback.
- ✓ Using GCP services and DevOps best practices.
- ✓ You will understand how all the pieces work together.

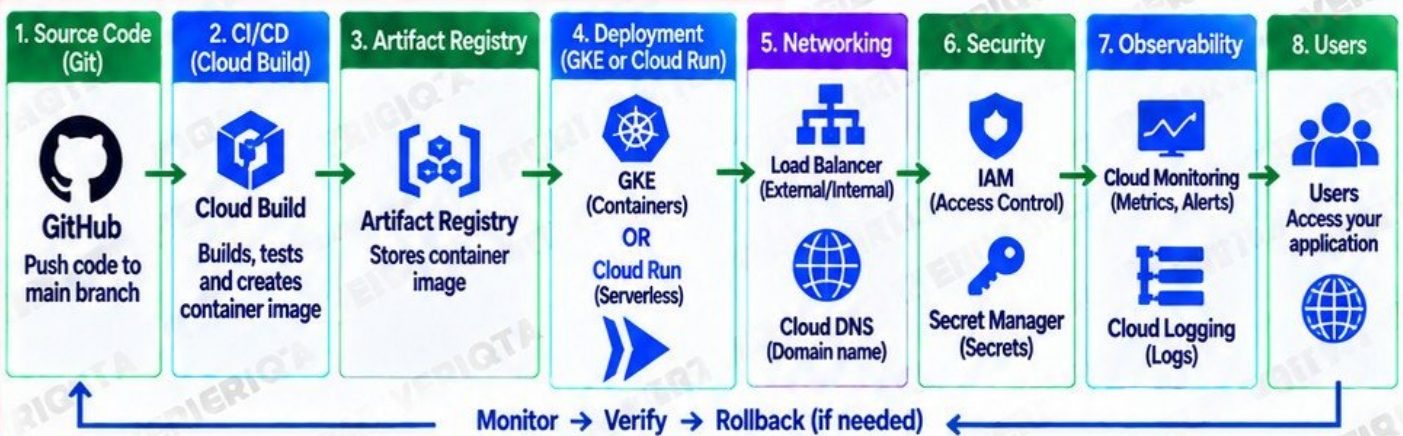
Key GCP Services Used

Git (Source Code)	IAM (Access Control)
Cloud Build (CI/CD)	Secret Manager (Secrets)
Artifact Registry (Container Images)	Cloud Monitoring (Metrics, Alerts)
GKE (Containers)	Cloud Logging (Logs)
Cloud Run (Serverless)	Cloud DNS (Domain Name)
VPC & Load Balancer (Networking)	Deployment Verification & Rollback

End-to-End Steps

- 1 Commit code to Git (e.g. GitHub).
- 2 Trigger CI/CD with Cloud Build.
- 3 Build and push container image to Artifact Registry.
- 4 Deploy to GKE or Cloud Run.
- 5 Configure networking (Load Balancer, DNS, firewall).
- 6 Use IAM for secure access.
- 7 Store and use secrets (Secret Manager).
- 8 Monitor with Cloud Monitoring and Cloud Logging.
- 9 Verify the deployment and test the application.
- 10 Rollback if issues are detected.

Production Deployment Architecture on GCP



Example: Cloud Build (cloudbuild.yaml)

```

steps:
- name: 'gcr.io/cloud-builders/docker'
  args: ['build', '-t', 'us-central1-docker.pkg.dev/$PROJECT_ID/my-app/my-app:$SHORT_SHA', '.']
- name: 'gcr.io/cloud-builders/docker'
  args: ['push', 'us-central1-docker.pkg.dev/$PROJECT_ID/my-app/my-app:$SHORT_SHA']
- name: 'gcr.io/cloud-builders/gke-deploy'
  args: ['run', '--filename=deployment.yaml', '--location=us-central1', '--cluster=my-cluster']
images:
- 'us-central1-docker.pkg.dev/$PROJECT_ID/my-app/my-app:$SHORT_SHA'
  
```

Example: Deployment to GKE

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
          image: us-central1-docker.pkg.dev/my-app/my-app:latest
          ports:
            - containerPort: 8888
  
```

Example: Deployment to Cloud Run

```

gcloud run deploy my-app \
  --image us-central1-docker.pkg.dev/my-app/my-app:latest \
  --region us-central1 \
  --allow-unauthenticated \
  --set-env-vars "ENV=production" \
  --set-secrets="DB_PASSWORD=projects/$PROJECT_ID/secrets/db-password:latest"
  
```

Example: Verify and Rollback

```

# Check deployment status (GKE)
kubectl rollout status deployment/my-app
# View logs
kubectl logs -l app=my-app --tail=100
# Rollback if needed
kubectl rollout undo deployment/my-app
  
```

Key Components and Their Roles

- Git: Source code and version control
- Cloud Build: CI/CD pipeline (build, test, deploy)
- Artifact Registry: Store container images
- GKE / Cloud Run: Run your application
- VPC, Load Balancer, DNS: Networking and access
- IAM: Secure access and permissions
- Secret Manager: Manage sensitive data
- Cloud Monitoring & Logging: Observe and troubleshoot

Best Practices

- ✓ Use a single, automated CI/CD pipeline.
- ✓ Store container images in Artifact Registry.
- ✓ Use IAM with least privilege.
- ✓ Keep secrets in Secret Manager (not in code).
- ✓ Use health checks and readiness probes.
- ✓ Set up monitoring, alerts and log-based metrics.
- ✓ Automate rollbacks for failed deployments.
- ✓ Test the entire flow in a non-production environment first.

Key Takeaways

- ✓ A production deployment is more than just deploying code.
- ✓ All components (CI/CD, security, networking, secrets, monitoring) work together.
- ✓ Design for failure and be ready to rollback.
- ✓ Use managed GCP services to reduce operational work.
- ✓ Monitoring and verification are essential to keep your application reliable.